23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : _____

Team Members :

Roll No	Student Name
AM.SC.U4CSE25245	Saharsh Baiju

Project Title : **O(1) Lite** — an ICPC practice tracker for college competitive-programming teams

3.1. Detailed description of the project

College teams preparing for the ICPC solve problems on Codeforces, follow practice problem lists, and usually track who solved what in a shared spreadsheet. That spreadsheet repeats every student's details and every problem's details on every row, so it is slow to update and easy to corrupt. **O(1) Lite** replaces it with a small relational database: students record their submissions, form ICPC teams of up to three, keep practice sheets for their team, and see a leaderboard and their progress on each sheet.

This is a deliberately compact design of **nine tables**, so that every table can be explained in a sentence and every normalisation step can be shown on real attributes.

Users

There is one kind of user, the **student**: they solve problems (submissions are recorded against them), belong to at most one team, and see the leaderboard and their team's sheet progress.

Why this needs a database, not just Codeforces API calls

Codeforces supplies only raw submissions for one handle at a time. Everything else the application shows either does not exist on Codeforces or needs data from many users at once:

- **Our own data:** universities, teams, practice sheets and topic tags are created in O(1). There is no API to fetch them from; the database is their only home.
- **Queries across users:** the leaderboard, a team's sheet progress, and "which sheet problems has nobody in the team solved" join and aggregate the submissions of many students.

Through the API that is one call per student on every page load, and the API is rate-limited; in SQL it is one query over stored rows.

- **History and availability:** submissions are synced once and stored, so pages work even when Codeforces is slow or down, and past data stays available.
- **Integrity:** keys, foreign keys, CHECK constraints and a trigger guarantee that a team never has four members, a sheet never lists a problem twice, and a submission never points at a missing user. An API gives no such guarantees about our data.
- **Derived values without duplication:** solved counts, ranks and progress are views computed from base tables, so they can never disagree with the submissions.

The Codeforces API is therefore only a *data source* for one table (Submission, via a sync job); the database is the system of record for everything.

What the system stores

- **University** — name (unique) and city.
- **User** — name, unique e-mail, password hash, optional Codeforces handle (unique), university, and the team they belong to (if any).
- **Team** — an ICPC team with a unique name; at most three students.
- **Problem** — identified by its Codeforces code (e.g. `1520C`), with a title and a difficulty rating. Its URL and difficulty band (Easy / Medium / Hard) are *derived* from the code and the rating, so they are not stored.
- **Topic** — Greedy, DP, Graphs, …; a problem can have many topics.
- **Submission** — one attempt by a user at a problem at a particular moment, with a verdict and a language.
- **Sheet** — a practice list belonging to one team, with an optional deadline, containing an ordered list of problems.

Business rules

1. Every user studies at exactly one university.
2. E-mail is unique; a Codeforces handle, if given, is unique.
3. A student belongs to at most one team; a team has at most three members (enforced by a trigger).
4. A problem is identified by its Codeforces code and may carry several topics.
5. A submission is identified by *who* submitted, *which problem* and *when*. A user cannot submit the same problem twice in the same instant.
6. A sheet belongs to exactly one team. Inside a sheet a problem appears once, and each position (1, 2, 3 …) is used once.
7. Values that can be computed (difficulty band, solved count, rank, sheet progress) are never stored; they are computed by views.

Scope compared with the full design

The earlier full design (`docs/DB-Design.pdf`, kept unchanged) had twenty relations. The lite version keeps the core loop — *problems → submissions → teams → sheets → leaderboard* — and removes features that add tables without adding new database concepts:

Removed / merged	**How the lite design handles it**
JudgeAccount (weak entity)	Merged into User as a single `cf_handle` column (only Codeforces is supported).
TeamMember	A student is in at most one team, so `team_id` is a nullable foreign key in User.
Contest, ContestProblem, Participation	Dropped. Ratings and contest history can be read live from the Codeforces API.
SheetAssignment, SheetCompletion	A sheet is assigned to a whole team; progress is computed from submissions.
Resource, Bookmark, CalendarEvent, Reminder	Dropped from the database (out of scope for the lite version).
Mentor role (team coach, sheet author)	Dropped for now: every user is a student, and a sheet belongs to its team.

3.2. ER Diagram

Chen notation: rectangles are entities, diamonds are relationships, ellipses are attributes (key attributes underlined, the derived attribute *difficulty* dashed). **Submission** is a **weak entity** (double rectangle): it has no key of its own and is identified through its owner User and its Problem plus the partial key *submitted_at*; *makes* and *for_problem* are therefore identifying relationships (double diamonds). Cardinalities are written on the edges, and a double line marks total participation.

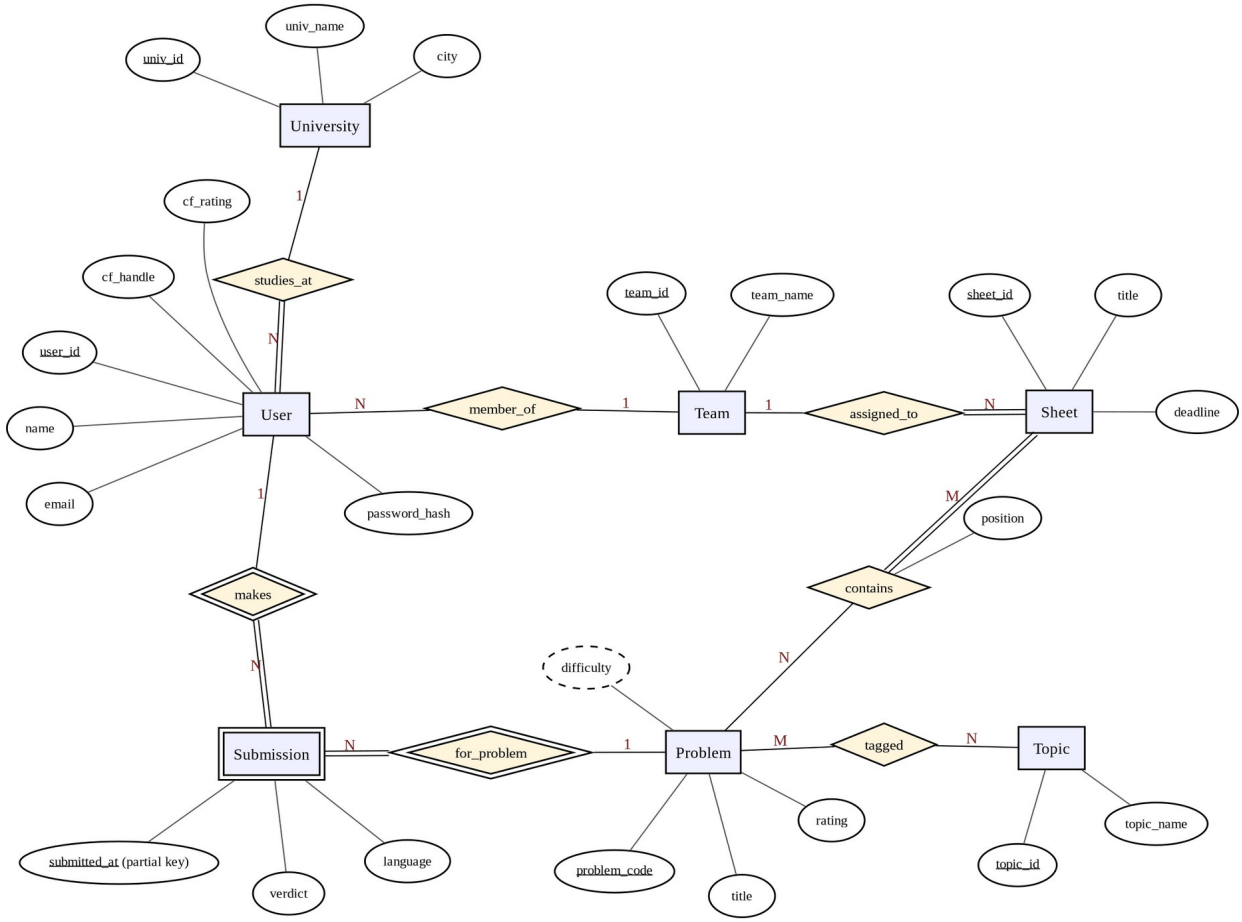


Figure 1 — ER diagram of O(1) Lite. Source: `docs/lite/er-lite.dot`.

Entity summary

Entity	Key	Other attributes
University	univ_id	univ_name, city
User	user_id	name, email, password_hash, cf_handle, cf_rating, cf_max_rating
Team	team_id	team_name
Problem	problem_code	title, rating, *difficulty (derived)*
Topic	topic_id	topic_name
Submission	owner user + problem + submitted_at	verdict, language

(weak)	(partial key)	
Sheet	sheet_id	title, deadline

Relationship summary

Relationship	Entities	Cardinality	Participation	Note
studies_at	User – University	N : 1	User total	—
member_of	User – Team	N : 1	both partial	≤ 3 students per team
assigned_to	Sheet – Team	N : 1	Sheet total	—
contains	Sheet – Problem	M : N	Sheet total	attribute *position*
tagged	Problem – Topic	M : N	both partial	—
makes *(identifying)*	User – Submission	1 : N	Submission total	—
for_problem *(identifying)*	Submission – Problem	N : 1	Submission total	—

3.3. Relational Schema

The ER diagram is converted to tables with the standard mapping rules:

- **Strong entity** → a table with the entity's key as primary key (University, User, Team, Problem, Topic, Sheet).
- **Weak entity** → a table whose primary key is the owners' keys plus the partial key: Submission(user_id, problem_code, submitted_at).
- **1 : N relationship** → the key of the "1" side is added as a foreign key on the "N" side (studies_at → User.univ_id, member_of → User.team_id, assigned_to → Sheet.team_id). Partial participation of the N side gives a nullable foreign key (User.team_id).
- **M : N relationship** → a separate junction table whose key is both keys, plus any relationship attribute (tagged → Problem_Topic, contains → Sheet_Problem with *position*).
- **Derived attribute** → not stored; computed in a view (§3.5).

Primary keys are underlined; → marks a foreign key and the table it references.

1. `University(univ_id, univ_name, city)`
2. `Team(team_id, team_name)`
3. `User(user_id, name, email, password_hash, cf_handle, cf_rating, cf_max_rating, univ_id → University, team_id → Team [nullable])`
4. `Topic(topic_id, topic_name)`
5. `Problem(problem_code, title, rating)`
6. `Problem_Topic(problem_code → Problem, topic_id → Topic)`
7. `Submission(user_id → User, problem_code → Problem, submitted_at, verdict, language)`
8. `Sheet(sheet_id, title, team_id → Team, deadline)`
9. `Sheet_Problem(sheet_id → Sheet, problem_code → Problem, position)`

Keys

Table	Primary key	Other candidate (alternate) keys	Foreign keys
University	univ_id	univ_name	—
Team	team_id	team_name	—
User	user_id	email (cf_handle is UNIQUE but nullable, so it is not a candidate key)	univ_id, team_id
Topic	topic_id	topic_name	—
Problem	problem_code	—	—
Problem_Topic	(problem_code, topic_id)	—	problem_code, topic_id
Submission	(user_id, problem_code, submitted_at)	—	user_id, problem_code
Sheet	sheet_id	—	team_id

Sheet_Problem	(sheet_id, problem_code)	(sheet_id, position)	sheet_id, problem_code

Every candidate key is also a **super key**; any super set of it, e.g. (user_id, name), is a super key but not a candidate key because it is not minimal.

Integrity constraints

Kind	**Where**	**Business rule**
Domain (CHECK)	800 ≤ rating ≤ 3500; verdict ∈ {OK, WRONG_ANSWER, …}; position > 0	4, 5
Entity integrity	every primary key is NOT NULL and unique	—
Key (UNIQUE)	email, cf_handle, univ_name, team_name, topic_name, (sheet_id, position)	2, 6
Referential integrity	all foreign keys; ON DELETE CASCADE for owned rows (a user's submissions, a sheet's problems), SET NULL when a team is deleted	5, 6
Trigger	`team_size_check` rejects a fourth member of a team	3

Indexes

Every PRIMARY KEY and UNIQUE constraint creates a B-tree index automatically. One extra index supports the "did you mean" college search on sign-up: `university_name_trgm`, a GIN index on *univ_name* using trigrams (the `pg_trgm` extension). It lets `similarity()` find "Amrita Vishwa Vidyapeetham" when a student types "amritha", which stops near-duplicate colleges from being inserted.

3.4. Normalization and Functional Dependency Analysis

We start from the spreadsheet the team used before the system existed and normalise it step by step. Every table in §3.3 comes out of this process.

3.4.1 Starting point: the unnormalised practice log

One row per submission; the *topics* cell holds a list.

user_id	name	email	univ_id	univ_name	team_id	team_name	problem_code	title	rating	topics	submitted_at
1	Saharsh	saharsh@am.edu	1	Amrita	1	Segfault Survivors	1520C	Not Adjacent Matrix	1000	Math	01-09 10:00
1	Saharsh	saharsh@am.edu	1	Amrita	1	Segfault Survivors	1520C	Not Adjacent Matrix	1000	Math	01-09 10:12
1	Saharsh	saharsh@am.edu	1	Amrita	1	Segfault Survivors	455A	Boredom	1500	DP	02-09 18:30
2	Anu	anu@am.edu	1	Amrita	1	Segfault Survivors	455A	Boredom	1500	DP	02-09 19:05
2	Anu	anu@am.edu	1	Amrita	1	Segfault Survivors	1547G	How Many Paths?	2100	DP, Graphs	03-09 21:40

This table suffers from all three anomalies:

- **Insertion anomaly** — a new problem cannot be recorded until someone submits it, and a new student cannot be recorded until they make a submission.
- **Update anomaly** — Saharsh's e-mail is stored three times; if the university is renamed, every row of every Amrita student must change. Missing one leaves the data inconsistent.
- **Deletion anomaly** — deleting Anu's only TLE submission also deletes the fact that problem 1547G exists and is a Graphs problem.

3.4.2 First Normal Form (1NF)

Definition: a relation is in 1NF if every attribute holds a single atomic value (no lists, no repeating groups). The *topics* column violates this.

Fix: a multi-valued attribute is moved to its own relation together with the key of the thing it describes. Topics describe a *problem*, so we create **Problem_Topic(<u>problem_code</u>, <u>topic</u>)** with one row per (problem, topic) pair — 1547G now gives two rows, (1547G, DP) and (1547G, Graphs). Later, topic names are moved to a **Topic(<u>topic_id</u>, topic_name)** lookup table so a name is typed once (renaming "DP" to "Dynamic Programming" becomes a one-row update).

What remains is the 1NF relation we analyse in the rest of this section:

```
PRACTICE_LOG(user_id, name, email, univ_id, univ_name, team_id, team_name,
```

```
problem_code, title, rating, submitted_at, verdict, language)
```

To keep the working short, each attribute gets a one-letter name:

U	**N**	**M**	**I**	**J**	**G**	**K**	**P**	**T**	**R**	**S**	**V**	**L**
user_id	name	email	univ_id	univ_name	team_id	team_name	problem_code	title	rating	submitted_at	verdict	language

3.4.3 Functional dependencies

Definition: X → Y (X functionally determines Y) if any two tuples that agree on X must also agree on Y. An FD is **trivial** if Y ⊆ X (e.g. UM → U), otherwise **non-trivial**. From the business rules, the spreadsheet's notes give this set F (written exactly as collected, including some redundancy that §3.4.6 removes):

#	**FD**	**Reason**
F1	`U → N M I G`	a user has one name, e-mail, university, team
F2	`M → U`	e-mail is unique
F3	`I → J`	a university id has one name
F4	`G → K`	a team id has one name
F5	`P → T R`	a problem has one title and rating
F6	`U P S → V L`	one user, one problem, one instant ⇒ one submission
F7	`U M → N`	noted separately on the sheet
F8	`U → J`	"the student's university name"
F9	`U P S → T`	"each submission row shows the problem title"

Armstrong's axioms are the rules for deriving new FDs from F:

- **Reflexivity:** if Y ⊆ X then X → Y. (UP → P)
- **Augmentation:** if X → Y then XZ → YZ. (P → T gives PS → TS)
- **Transitivity:** if X → Y and Y → Z then X → Z. (U → I, I → J gives U → J)
- Derived rules: **union** (X → Y, X → Z ⇒ X → YZ), **decomposition** (X → YZ ⇒ X → Y), **pseudo-transitivity** (X → Y, WY → Z ⇒ WX → Z).

3.4.4 Attribute closure and candidate keys

Definition: the closure X^+ is the set of all attributes that X determines. **Algorithm:** start with result = X; repeatedly, for every FD A → B in F with A ⊆ result, add B to result; stop when nothing changes. X is a **super key** if X^+ contains every attribute, and a **candidate key** if, in addition, no proper subset of X is a super key.

{U, P, S}$^+$

```
start : {U, P, S}
F1 U → NMIG : {U, P, S, N, M, I, G}
F3 I → J : {U, P, S, N, M, I, G, J}
F4 G → K : {U, P, S, N, M, I, G, J, K}
F5 P → TR : {U, P, S, N, M, I, G, J, K, T, R}
F6 UPS → VL : {U, P, S, N, M, I, G, J, K, T, R, V, L} = all 13 attributes ✓ super key
```

{M, P, S}$^+$

```
start {M, P, S} → F2 M → U adds U → now {U, P, S} ⊆ result, so (as above) all 13 attributes ✓ super key
```

Are they minimal?

X	X^+	Super key?
{U}	{U, N, M, I, G, J, K}	no
{M}	{M, U, N, I, G, J, K}	no
{U, P}	{U, P, N, M, I, G, J, K, T, R}	no — missing S, V, L
{U, S}	{U, S, N, M, I, G, J, K}	no
{P, S}	{P, S, T, R}	no

A quick way to see this: **P and S never appear on the right-hand side of any FD**, so they must be in every key. {P, S} alone is not enough, and adding either U or M completes it. Hence the **candidate keys are {U, P, S} and {M, P, S}**. We choose {U, P, S} as the primary key.

- **Prime attributes** (in some candidate key): U, M, P, S.
- **Non-prime attributes**: N, I, J, G, K, T, R, V, L.

3.4.5 Closure of a set of FDs (F^+)

Definition: F^+ is the set of *all* FDs that can be derived from F using Armstrong's axioms. It is far too large to list (it includes every trivial FD over 13 attributes), so in practice we use two techniques:

(a) Deriving members of F^+ with the axioms:

Derived FD	Derivation

U → J	U → I (F1, decomposition), I → J (F3) ⇒ transitivity
M → N	M → U (F2), U → N (F1, decomposition) ⇒ transitivity
M → K	M → U, U → G, G → K ⇒ transitivity twice
U P → T	P → T (F5) ⇒ augmentation with U gives UP → TU ⇒ decomposition
M P S → V L	M → U (F2) and UPS → VL (F6) ⇒ pseudo-transitivity

(b) Membership test using attribute closure: X → Y is in F^+ exactly when Y ⊆ X^+.

Is … in F^+?	**Compute**	**Answer**
M → K	{M}+ = {M,U,N,I,G,J,K} ∋ K	Yes
I → U	{I}+ = {I, J} ∌ U	No — many students share a university
G → I	{G}+ = {G, K} ∌ I	No — the data does not force a team's university

Two FD sets F and G are **equivalent** ($F^+ = G^+$) if every FD of G is in F^+ and every FD of F is in G^+; this is how we check that the minimal cover below says the same thing as F.

3.4.6 Minimal (canonical) cover

Definition: a minimal cover F_c is an FD set equivalent to F in which (1) every right-hand side is a single attribute, (2) no left-hand side has an extraneous attribute, and (3) no FD is redundant.

Step 1 — single attribute on the right. Splitting F gives 14 FDs:

`U→N, U→M, U→I, U→G, M→U, I→J, G→K, P→T, P→R, UPS→V, UPS→L, UM→N, U→J, UPS→T`

Step 2 — remove extraneous left-hand attributes. In XA → B, A is extraneous if $B \in X^+$ (computed with F).

FD	Try removing	Closure	Result
`UM → N`	M	`{U}+ ∋ N`	M extraneous → `U → N` (already present, drop)
`UPS → T`	U, then S	`{PS}+ ∋ T, then {P}+ ∋ T`	→ `P → T` (already present, drop)
`UPS → V`	U / P / S	`{PS}+, {US}+, {UP}+ — none contain V`	nothing extraneous
`UPS → L`	U / P / S	`same, none contain L`	nothing extraneous

> In UM → N, U is *also* extraneous on its own (M → N holds via M → U → N). Removing U first instead of M would end with M → N in place of U → N. Both answers are correct: **a minimal cover is not unique**. We keep U → N because it reads naturally.

Step 3 — remove redundant FDs. A → B is redundant if $B \in A^+$ computed *without* that FD.

FD	Closure of LHS without it	Redundant ?
`U → J`	`{U}+ = {U,N,M,I,G,J,K} — J reached via I → J`	**Yes, remove**
`U → N`	`{U}+ = {U,M,I,G,J,K}`	No
`U → M`	`{U}+ = {U,N,I,G,J,K}`	No
`M → U`	`{M}+ = {M}`	No
`U → I, U → G, I → J, G → K, P → T, P → R, UPS → V, UPS → L`	the right-hand attribute is not reachable without the FD itself	No

Minimal cover (written with the union rule for readability):

F_c = { U → N M I G, M → U, I → J, G → K, P → T R, U P S → V L }

Equivalence check: every FD of F_c is in F, and the three removed FDs (UM → N, U → J, UPS → T) were each shown above to follow from the rest, so $F_c^+ = F^+$.

3.4.7 Second Normal Form (2NF)

Definition: a relation is in 2NF if it is in 1NF and no **non-prime** attribute is **partially dependent** on a candidate key (i.e. depends on only part of a composite key).

PRACTICE_LOG has key {U, P, S}. Checking F_C:

FD	Analysis	2NF?
`U → N M I G`	U is a proper part of the key; N, I, G are non-prime	**Violation** (partial)
`P → T R`	P is a proper part of the key; T, R non-prime	**Violation** (partial)
`M → U`	U is prime, so this does not violate 2NF	OK
`U P S → V L`	full dependency on the whole key	OK

Decomposition: for each partial dependency X → Y, take out X^+ as its own relation.

```
R1 = USER_INFO(U, N, M, I, J, G, K)   ← {U}+
R2 = PROBLEM(P, T, R)   ← {P}+
R3 = SUBMISSION(U, P, S, V, L)   ← key + what depends on the whole key
```

R1 and R2 have single-attribute keys, so they cannot have partial dependencies; in R3, V and L depend on the whole key. All three are in 2NF. The insertion anomaly for problems is gone: a problem can now exist with no submissions.

3.4.8 Third Normal Form (3NF)

Definition: a relation is in 3NF if for every non-trivial FD X → A, either **X is a super key** or **A is a prime attribute**. Equivalently: it is in 2NF and no non-prime attribute is **transitively dependent** on a key.

R1 = USER_INFO(U, N, M, I, J, G, K) violates 3NF: I → J and G → K hold, but neither I nor G is a super key and J, K are non-prime. These are transitive dependencies U → I → J and U → G → K — the source of the "rename the university in every row" update anomaly.

Decomposition of R1:

```
R1a = USER(U, N, M, I, G)   FDs: U → NMIG, M → U
R1b = UNIVERSITY(I, J)   FDs: I → J
R1c = TEAM(G, K)   FDs: G → K
```

Now every determinant in every relation is a key: in USER, U and M are both candidate keys; in UNIVERSITY, TEAM, PROBLEM and SUBMISSION the only determinant is the primary key. All relations are in 3NF.

Cross-check with the 3NF synthesis algorithm

The textbook synthesis algorithm builds one relation per FD of F_C — (U,N,M,I,G), (M,U), (I,J), (G,K), (P,T,R), (U,P,S,V,L) — drops (M,U) because it is contained in (U,N,M,I,G), and adds a key relation only if no relation already contains a candidate key ((U,P,S) is inside SUBMISSION, so none is added). The result is exactly the decomposition above. Synthesis is guaranteed to be **lossless**

and dependency preserving; §3.4.10 and §3.4.11 prove both directly.

3.4.9 BCNF check (beyond 3NF)

BCNF is stricter: for every non-trivial $X \rightarrow A$, X must be a super key (no exception for prime A). Every determinant in our decomposition is a candidate key, so all relations are also in **BCNF**.

3.4.10 Lossless-join decomposition

Definition: decomposing R into R1 … Rn is lossless if joining the pieces always gives back exactly R — no tuples lost and, more importantly, no **spurious** tuples added.

Binary test: R → {R1, R2} is lossless if R1 ∩ R2 → R1 or R1 ∩ R2 → R2 is in F^+ (the common attributes form a key of one side). Every split we made passes it:

Split	**R1 ∩ R2**	**Test**
PRACTICE_LOG → USER_INFO + (U,P,S,T,R,V,L)	{U}	U → USER_INFO ✓
(U,P,S,T,R,V,L) → PROBLEM + SUBMISSION	{P}	P → PROBLEM ✓
USER_INFO → (U,N,M,I,G,K) + UNIVERSITY	{I}	I → UNIVERSITY ✓
(U,N,M,I,G,K) → USER + TEAM	{G}	G → TEAM ✓

Tableau (chase) test for the whole decomposition at once. Build one row per relation, writing **a** in the columns the relation has and b_i elsewhere. Apply the FDs: whenever two rows agree on the left side of an FD, make their right-side values equal (prefer **a**). If some row becomes all **a**, the decomposition is lossless.

Initial tableau:

	U	**N**	**M**	**I**	**J**	**G**	**K**	**P**	**T**	**R**	**S**	**V**	**L**
USER	**a**	**a**	**a**	**a**	b_1	**a**	b_1	b_1	b_1	b_1	b_1	b_1	b_1
UNIVERSITY	b_2	b_2	b_2	**a**	**a**	b_2	b_2	b_2	b_2	b_2	b_2	b_2	b_2
TEAM	b_3	b_3	b_3	b_3	b_3	**a**	**a**	b_3	b_3	b_3	b_3	b_3	b_3
PROBLEM	b_4	b_4	b_4	b_4	b_4	b_4	b_4	**a**	**a**	**a**	b_4	b_4	b_4
SUBMISSION	**a**	b_5	b_5	b_5	b_5	b_5	b_5	**a**	b_5	b_5	**a**	**a**	**a**

Steps: USER and SUBMISSION agree on U, so U → NMIG copies **a** into N, M, I, G of SUBMISSION. Now USER, UNIVERSITY and SUBMISSION agree on I, so I → J gives J = **a**; likewise G → K gives K = **a**. PROBLEM and SUBMISSION agree on P, so P → TR gives T, R = **a**.

Final tableau (underlined = changed by the chase):

	U	**N**	**M**	**I**	**J**	**G**	**K**	**P**	**T**	**R**	**S**	**V**	**L**
USER	**a**	**a**	**a**	**a**	**a**	**a**	**a**	b_1	b_1	b_1	b_1	b_1	b_1
UNIVERSITY	b_2	b_2	b_2	**a**	**a**	b_2	b_2	b_2	b_2	b_2	b_2	b_2	b_2
TEAM	b_3	b_3	b_3	b_3	b_3	**a**	**a**	b_3	b_3	b_3	b_3	b_3	b_3
PROBLEM	b_4	b_4	b_4	b_4	b_4	b_4	b_4	**a**	**a**	**a**	b_4	b_4	b_4
SUBMISSION	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**	**a**

The SUBMISSION row is all **a**, so the decomposition is **lossless**. ✓

Counter-example: a lossy decomposition

Suppose SUBMISSION(U, P, S, V, L) were split into A(U, P) and B(P, S, V, L). The common

attribute is P, and P is a key of neither A nor B, so the test fails. With the sample data:

SUBMISSION (original)				
U	**P**	**S**	**V**	**L**
1	455A	02-09 18:30	OK	C++
2	455A	02-09 19:05	OK	Py

A = {(1, 455A), (2, 455A)}, B = {(455A, 18:30, OK, C++), (455A, 19:05, OK, Py)}. A ⋈ B has **4** rows, including (1, 455A, 19:05, OK, Py) — a submission Saharsh never made. These **spurious tuples** mean information has been lost: we can no longer tell who submitted when.

3.4.11 Dependency preservation

Definition: a decomposition R1 … Rn is dependency preserving if $(F_1 \cup F_2 \cup \ldots \cup F_n)^+ = F^+$, where F_i is the projection of F^+ onto R_i (the FDs that use only attributes of R_i). If it holds, every constraint can be checked inside a single table, without joins.

Relation	**Projected FDs F_i**	**FDs of F_c covered**
USER(U,N,M,I,G)	`U → NMIG, M → U`	U → NMIG, M → U
UNIVERSITY(I,J)	`I → J`	I → J
TEAM(G,K)	`G → K`	G → K
PROBLEM(P,T,R)	`P → TR`	P → TR
SUBMISSION(U,P,S,V,L)	`UPS → VL`	UPS → VL

Each FD of F_c lies completely inside one relation, so the union of the projections is F_c itself, whose closure equals F^+. The decomposition is **dependency preserving**. ✓

Counter-example: a decomposition that loses a dependency

Suppose USER_INFO(U, N, M, I, J, G, K) had been split into X(U, N, M, I, G, K) and Y(U, J) (the university name stored against each user). It is lossless (U is a key of both), but I → J is no longer inside any single table. Testing it with the preservation algorithm:

```
result = {I}
on X: ({I} ∩ X)⁺ ∩ X = {I, J} ∩ X = {I} → no change
on Y: ({I} ∩ Y)⁺ ∩ Y = ∅ → no change
J ∉ result ⇒ I → J is not preserved
```

The database could then store two different names for the same univ_id and nothing would stop it short of joining X and Y on every insert. Our chosen decomposition avoids this.

3.4.12 Final tables and their normal forms

The final schema (§3.3) adds a few attributes that do not change the analysis — password_hash, cf_handle, cf_rating and cf_max_rating on User, city on University — and the Topic, Problem_Topic, Sheet and Sheet_Problem tables. Their FDs:

Table	**Non-trivial FDs**	**Every determinant a key?**	**Normal form**
University	`univ_id → univ_name, city` `univ_name → univ_id`	yes	BCNF
Team	`team_id → team_name` `team_name → team_id`	yes	BCNF

User	`user_id → name, email, password_hash,`
	`cf_handle, cf_rating, cf_max_rating,`
	`univ_id, team_id`
		yes	BCNF

	`email → user_id`		
Topic	`topic_id → topic_name` `topic_name → topic_id`	yes	BCNF
Problem	`problem_code → title, rating`	yes	BCNF
Problem_Topic	none (all-key relation)	—	BCNF
Submission	`user_id, problem_code, submitted_at → verdict, language`	yes	BCNF
Sheet	`sheet_id → title, team_id, deadline`	yes	BCNF
Sheet_Problem	`sheet_id, problem_code → position` `sheet_id, position → problem_code`	yes (two overlapping candidate keys)	BCNF

> **Design decision worth noting:** a problem's URL and difficulty band are functions of problem_code and rating; storing them would add the FDs problem_code → url and rating → difficulty, the second of which is a transitive dependency. They are therefore derived, not stored. The Codeforces rating, by contrast, *is* stored (cf_rating, cf_max_rating): it cannot be computed from our tables, because it comes from Codeforces' own contest history. It is a snapshot refreshed on every sync, and depends only on user_id, so User stays in BCNF.

3.5. Appendix — SQL implementation

PostgreSQL DDL for the nine tables, the team-size trigger, and the views that compute derived values (`docs/lite/schema-lite.sql`). Sample data is in `seed-lite.sql`. Tested on PostgreSQL.

```
-- O(1) Lite — ICPC practice tracker, relational schema (PostgreSQL).
-- 9 tables, every one in 3NF (and in fact BCNF). See DB-Design-Lite.pdf §3.4.
-- Derived values (difficulty, solved count, leaderboard, sheet progress) are
-- NOT stored; the views at the bottom compute them from the base tables.

BEGIN;

-- pg_trgm gives similarity() for "did you mean" college search on sign-up.
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 1. University ---------------------------------------------------------
CREATE TABLE university (
  univ_id    SERIAL       PRIMARY KEY,
  univ_name  VARCHAR(120) NOT NULL UNIQUE,
  city       VARCHAR(60)  NOT NULL
);

-- Trigram index: makes "names similar to what the user typed" fast.
CREATE INDEX university_name_trgm ON university USING gin (univ_name gin_trgm_ops);

-- 2. Team ---------------------------------------------------------------
CREATE TABLE team (
  team_id    SERIAL      PRIMARY KEY,
  team_name  VARCHAR(60) NOT NULL UNIQUE
);

-- 3. User ---------------------------------------------------------------
CREATE TABLE app_user (
  user_id        SERIAL       PRIMARY KEY,
  name           VARCHAR(80)  NOT NULL,
  email          VARCHAR(254) NOT NULL UNIQUE,          -- alternate (candidate) key
  password_hash  VARCHAR(255) NOT NULL,                 -- werkzeug scrypt hash, never the password
  cf_handle      VARCHAR(40)  UNIQUE,                   -- Codeforces handle, optional
  cf_rating      INT,                                   -- current Codeforces rating, refreshed on
sync
  cf_max_rating  INT,                                   -- best Codeforces rating
  univ_id        INT          NOT NULL REFERENCES university(univ_id),
  team_id        INT          REFERENCES team(team_id) ON DELETE SET NULL   -- NULL = no team
);

-- 4. Topic --------------------------------------------------------------
CREATE TABLE topic (
  topic_id    SERIAL      PRIMARY KEY,
  topic_name  VARCHAR(40) NOT NULL UNIQUE
);

-- 5. Problem (problem_code is the Codeforces code, e.g. '1520C';
--    the URL is derived from it, so it is not stored)
CREATE TABLE problem (
  problem_code  VARCHAR(12)  PRIMARY KEY,
  title         VARCHAR(200) NOT NULL,
  rating        INT          CHECK (rating BETWEEN 800 AND 3500)
```

```
);

-- 6. Problem ↔ Topic (M:N) ------------------------------------------------
CREATE TABLE problem_topic (
  problem_code  VARCHAR(12) REFERENCES problem(problem_code) ON DELETE CASCADE,
  topic_id      INT         REFERENCES topic(topic_id),
  PRIMARY KEY (problem_code, topic_id)
);

-- 7. Submission: one attempt by one user at one problem at one instant -----
CREATE TABLE submission (
  user_id       INT         REFERENCES app_user(user_id) ON DELETE CASCADE,
  problem_code  VARCHAR(12) REFERENCES problem(problem_code),
  submitted_at  TIMESTAMP   NOT NULL,
  verdict       VARCHAR(25) NOT NULL CHECK (verdict IN
                  ('OK', 'WRONG_ANSWER', 'TIME_LIMIT_EXCEEDED', 'RUNTIME_ERROR',
                   'MEMORY_LIMIT_EXCEEDED', 'COMPILATION_ERROR')),
  language      VARCHAR(50) NOT NULL,
  PRIMARY KEY (user_id, problem_code, submitted_at)
);

-- 8. Practice sheet: an ordered problem list for one team -----------------
CREATE TABLE sheet (
  sheet_id    SERIAL       PRIMARY KEY,
  title       VARCHAR(120) NOT NULL,
  team_id     INT          NOT NULL REFERENCES team(team_id) ON DELETE CASCADE,
  deadline    DATE
);

-- 9. Sheet ↔ Problem (M:N) with an order ----------------------------------
CREATE TABLE sheet_problem (
  sheet_id      INT         REFERENCES sheet(sheet_id) ON DELETE CASCADE,
  problem_code  VARCHAR(12) REFERENCES problem(problem_code),
  position      SMALLINT    NOT NULL CHECK (position > 0),
  PRIMARY KEY (sheet_id, problem_code),
  UNIQUE (sheet_id, position)                          -- second candidate key
);

-- Trigger: ICPC teams have at most three members --------------------------
CREATE FUNCTION check_team_size() RETURNS trigger AS $$
BEGIN
  IF NEW.team_id IS NOT NULL AND
     (SELECT count(*) FROM app_user
       WHERE team_id = NEW.team_id AND user_id <> NEW.user_id) >= 3 THEN
    RAISE EXCEPTION 'team % already has 3 members', NEW.team_id;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER team_size_check
  BEFORE INSERT OR UPDATE OF team_id ON app_user
  FOR EACH ROW EXECUTE FUNCTION check_team_size();

-- ====================================================================== views
-- Difficulty band is a function of rating → derived, not stored.
CREATE VIEW v_problem_difficulty AS
SELECT problem_code, title, rating,
       CASE WHEN rating IS NULL THEN NULL          -- unrated (new) problem
            WHEN rating < 1400 THEN 'Easy'
```

```
            WHEN rating < 2000 THEN 'Medium'
            ELSE 'Hard' END AS difficulty
FROM problem;

-- Leaderboard: distinct problems solved per user.
CREATE VIEW v_leaderboard AS
SELECT u.user_id, u.name, un.univ_name,
       count(DISTINCT s.problem_code) AS solved
FROM app_user u
JOIN university un ON un.univ_id = u.univ_id
LEFT JOIN submission s ON s.user_id = u.user_id AND s.verdict = 'OK'
GROUP BY u.user_id, u.name, un.univ_name;

-- Sheet progress: for every member of the sheet's team, how many sheet problems are solved.
CREATE VIEW v_sheet_progress AS
SELECT sh.sheet_id, u.user_id, u.name,
       count(sp.problem_code) AS total,
       count(sp.problem_code) FILTER (WHERE EXISTS (
         SELECT 1 FROM submission s
         WHERE s.user_id = u.user_id AND s.problem_code = sp.problem_code
           AND s.verdict = 'OK')) AS solved
FROM sheet sh
JOIN app_user u       ON u.team_id = sh.team_id
JOIN sheet_problem sp ON sp.sheet_id = sh.sheet_id
GROUP BY sh.sheet_id, u.user_id, u.name;

COMMIT;
```

Sample queries

From `docs/lite/queries-lite.sql`; each was run against the sample data.

```
-- Q1 (JOIN): every accepted submission with the student's and the problem's details.
SELECT u.name, p.problem_code, p.title, s.submitted_at
FROM submission s
JOIN app_user u ON u.user_id = s.user_id
JOIN problem  p ON p.problem_code = s.problem_code
WHERE s.verdict = 'OK'
ORDER BY s.submitted_at;

-- Q2 (GROUP BY + HAVING): students who have solved at least 2 distinct problems.
SELECT u.name, count(DISTINCT s.problem_code) AS solved
FROM app_user u JOIN submission s ON s.user_id = u.user_id
WHERE s.verdict = 'OK'
GROUP BY u.name
HAVING count(DISTINCT s.problem_code) >= 2;

-- Q3 (NOT EXISTS subquery): problems on sheet 1 that user 3 (Rahul) has not solved yet.
SELECT sp.position, sp.problem_code
FROM sheet_problem sp
WHERE sp.sheet_id = 1
  AND NOT EXISTS (SELECT 1 FROM submission s
                  WHERE s.user_id = 3 AND s.problem_code = sp.problem_code
                    AND s.verdict = 'OK')
ORDER BY sp.position;

-- Q4 (M:N through a junction table): how many problems each topic has, and their average rating.
SELECT t.topic_name, count(*) AS problems, round(avg(p.rating)) AS avg_rating
FROM topic t
JOIN problem_topic pt ON pt.topic_id = t.topic_id
JOIN problem p        ON p.problem_code = pt.problem_code
GROUP BY t.topic_name
ORDER BY problems DESC, t.topic_name;

-- Q5 (VIEW): the leaderboard, ranked.
SELECT rank() OVER (ORDER BY solved DESC) AS rank, name, univ_name, solved
FROM v_leaderboard;

-- Q6 (LEFT JOIN): every team with its member count, including empty teams.
SELECT t.team_name, count(u.user_id) AS members
FROM team t
LEFT JOIN app_user u ON u.team_id = t.team_id
GROUP BY t.team_name
ORDER BY members DESC;
```